

Task 2 – Dynamic Prompt Builder Service

Objective

The objective of this task was to understand Dynamic Prompt Engineering and Prompt Structuring by creating a service that converts structured metadata into optimized LLM-ready prompts.

Problem Statement

Raw text prompts are often inconsistent and difficult to maintain. To solve this problem, a Dynamic Prompt Builder Service was developed that accepts structured JSON input and automatically generates a standardized prompt template suitable for AI models.

Technologies Used

- Python
 - FastAPI
 - HTML
 - CSS
 - JavaScript
 - Fetch API
-

Implementation

Backend Development

A FastAPI backend was created with a POST endpoint named `/build-prompt`.

The endpoint accepts a JSON payload containing:

```
{  
  "project": "Inventory System",  
  "skills": ["Python", "FastAPI"]  
}
```

The backend processes the data and dynamically constructs a prompt using string templates.

Example generated prompt:

Generate ATS friendly bullet points.

Project:

Inventory System

Skills:

Python, FastAPI

The generated prompt is returned as a JSON response.

Frontend Development

A modern user interface was developed using HTML, CSS, and JavaScript.

Features implemented:

- Responsive design
- Modern glassmorphism UI
- User-friendly form inputs
- API integration using Fetch API
- Dynamic display of generated prompts
- Loading animation during API requests

The frontend collects project details and skills from the user and sends them to the FastAPI backend.

API Workflow

1. User enters project information.
 2. User enters required skills.
 3. Frontend sends a POST request to the FastAPI endpoint.
 4. Backend generates a structured prompt.
 5. Generated prompt is returned as JSON.
 6. Frontend displays the generated prompt to the user.
-

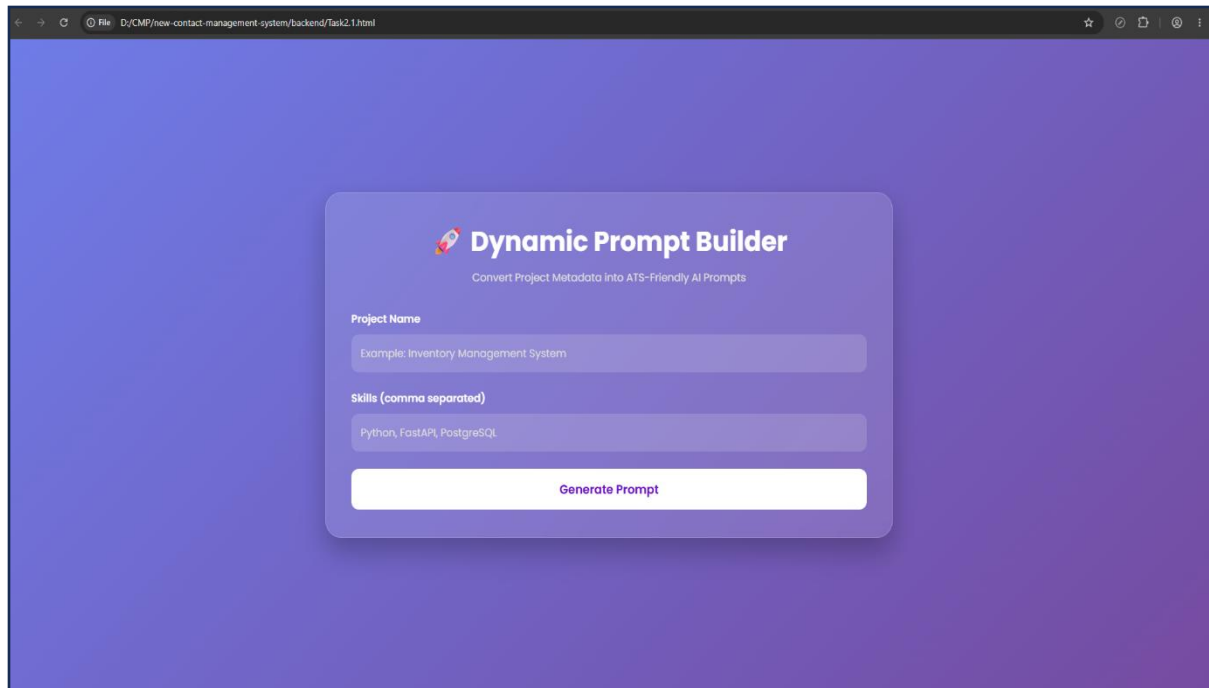
Learning Outcomes

Through this task, the following concepts were learned:

- Dynamic prompt generation
- Prompt engineering fundamentals
- FastAPI endpoint creation

- JSON request and response handling
 - Frontend and backend communication
 - Fetch API integration
 - Template-based string construction
 - Building AI-ready context from structured metadata
-

Screenshot 1: Frontend User Interface



The screenshot shows a web browser window with the address bar displaying "D:\CMP\new-contact-management-system\backend\Task2.1.html". The main content area has a purple gradient background. In the center, there is a white rounded rectangle with a shadow. At the top of this rectangle is a rocket icon followed by the text "Dynamic Prompt Builder" and a subtitle "Convert Project Metadata into ATS-Friendly AI Prompts". Below this, there are two input fields: "Project Name" with the example text "Example: Inventory Management System" and "Skills (comma separated)" with the example text "Python, FastAPI, PostgreSQL". At the bottom of the white rectangle is a white button with the text "Generate Prompt".

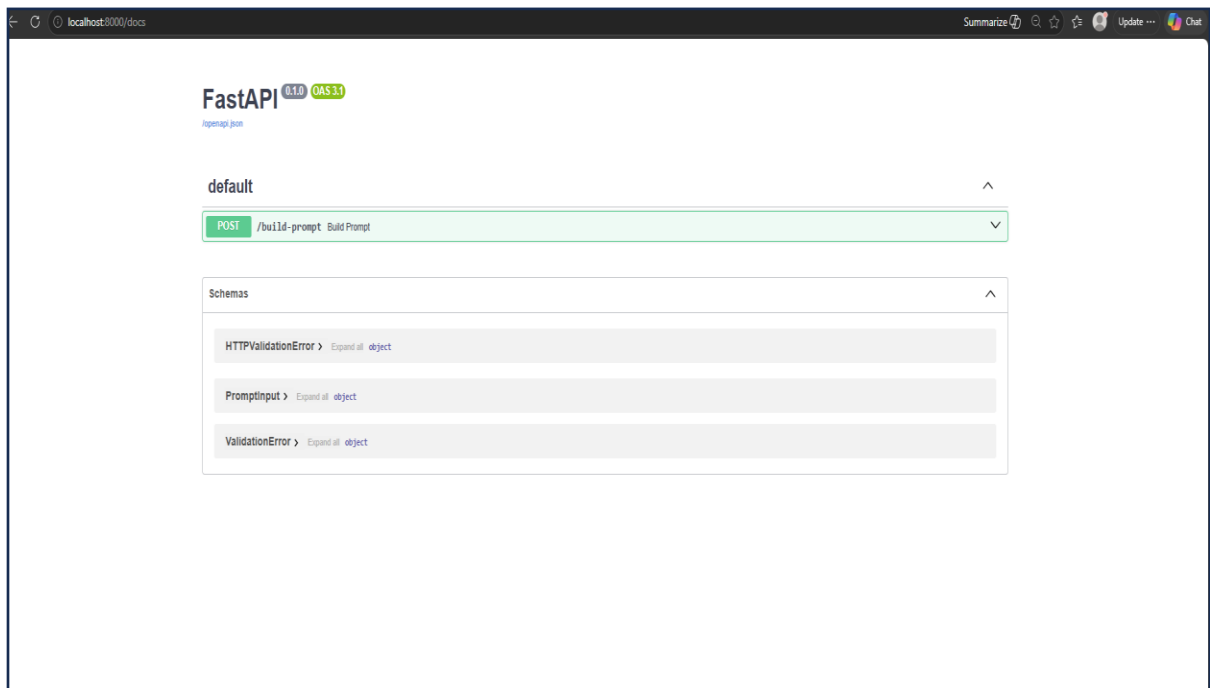
Screenshot 2 : FastAPI Source Code (Backend)

```
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  from typing import List
4  from fastapi.middleware.cors import CORSMiddleware
5
6  app = FastAPI()
7
8  app.add_middleware(
9      CORSMiddleware,
10     allow_origins=["*"],
11     allow_credentials=True,
12     allow_methods=["*"],
13     allow_headers=["*"],
14 )
15
16 class PromptInput(BaseModel):
17     project: str
18     skills: List[str]
19
20 @app.post("/build-prompt")
21 def build_prompt(data: PromptInput):
22
23     prompt = f"""Generate ATS friendly bullet points.
24
25     Project:
26     {data.project}
27
28     Skills:
29     {", ".join(data.skills)}
30     """
31
32     return {
33         "prompt": prompt
34     }
```

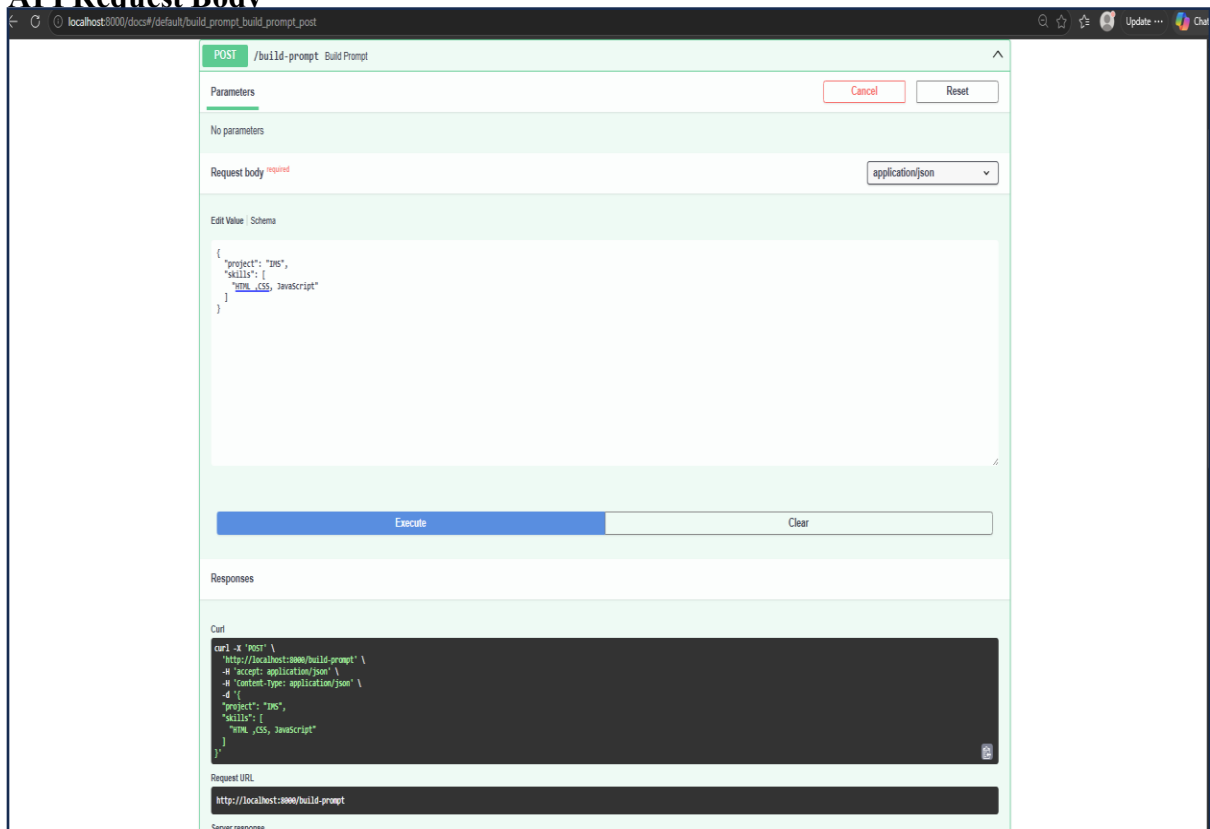
(Frontend Code)

```
2  <html lang="en">
167 <body>
209 <script>
211 async function generatePrompt(){
220
224     const response = await fetch(
225         "http://127.0.0.1:8000/build-prompt",
226         {
227             method:"POST",
228             headers:{
229                 "Content-Type":"application/json"
230             },
231             body:JSON.stringify({
232                 project:project,
233                 skills:skills
234             })
235         }
236     );
237
238     const data = await response.json();
239
240     document.getElementById("output").textContent =
241         data.prompt;
242
243     document.getElementById("outputCard")
244         .style.display="block";
245
246     }
247 catch(error){
248
249     document.getElementById("output").textContent =
250         "Error connecting to API";
251
252     document.getElementById("outputCard")
253         .style.display="block";
254
255     }
256     document.getElementById("loader").style.display="none";
257 }
```

Screenshot 3 : API Endpoint Testing using FastAPI Swagger UI



API Request Body



API Responses

The screenshot shows a web browser window with the address bar displaying `localhost:3000/docs#/default/build_prompt_build_prompt_post`. The page content is divided into several sections:

- curl:** A terminal window showing the command used to make the request:

```
curl -X 'POST' \
  http://localhost:3000/build-prompt/ \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "project": "IMS",
    "skills": [
      "HTML", "CSS", "JavaScript"
    ]
  }'
```
- Request URL:** `http://localhost:3000/build-prompt`
- Server response:** A table with two columns: **Code** and **Details**.
 - 200:** The response body is a JSON object: `{ "prompt": "Generate ATS friendly bullet points.\n\nproject:IMS\n\nskills:HTML ,CSS, JavaScript"`. There is a **Download** button next to it.
 - Response headers:** A list of headers: `access-control-allow-credentials: true, access-control-allow-origin: http://localhost:3000, content-length: 182, content-type: application/json, date: Mon, 15 Jun 2026 06:08:17 GMT, server: nginx, vary: origin`.
- Responses:** A table with three columns: **Code**, **Description**, and **Links**.
 - 200:** Successful Response. Media type: `application/json`. Controls accept header. Example Value: `"string"`. No links.
 - 422:** Validation Error. Media type: `application/json`. Example Value: `"string"`. No links.

Generated Output Displayed

The screenshot shows the **Dynamic Prompt Builder** web application. The interface is as follows:

- Header:** **Dynamic Prompt Builder** with a rocket icon and the tagline "Convert Project Metadata Into ATS-Friendly AI Prompts".
- Form:**
 - Project Name:** A text input field containing "Inventory Management System".
 - Skills (comma separated):** A text input field containing "HTML, CSS, JavaScript".
 - Generate Prompt:** A button with a white background and purple text.
- Generated Prompt:** A dark blue box containing the following text:

```
Generate ATS friendly bullet points.

Project:
Inventory Management System

Skills:
HTML, CSS, JavaScript
```

Challenges Faced

- Configuring communication between frontend and backend.
- Handling Cross-Origin Resource Sharing (CORS) issues.
- Formatting dynamic data into a consistent prompt structure.
- Displaying API responses dynamically on the frontend.

Conclusion

This task provided hands-on experience in prompt engineering, API development, and frontend-backend integration. The developed solution effectively transforms structured input data into reusable and consistent LLM-ready prompts, improving prompt quality and maintainability for AI-based applications.